

APIOBike

APIOBike 是一項在 APIO 市新推出的共享單車服務。全市設置了多個站點，使用者可以在任何站點借還單車。由於方便又低成本，它很快就成為 APIO 市最重要的交通工具。然而，APIOBike 遇到了一個所有共享單車服務都常見的問題：不同站點之間的借車與還車比率失衡。這導致有些站點幾乎沒有單車，附近居民想借車卻借不到，而其他站點則堆滿了當地居民用不到的單車。

為了解決這個問題，APIOBike 計畫在每晚派出一輛調度卡車，在各站點之間重新分配單車，確保每個站點都維持適當數量的單車。APIO 市共有 N 個站點，編號為 $0, 1, \dots, N-1$ 。經過觀察，APIOBike 發現每晚站點 i 的單車數量固定為 $A[i]$ ，而且夜間不會有人借車或還車。公司希望每天早上時，站點 i 恰好擁有 $B[i]$ 輛單車。

在這 N 個站點之中，有 $N-1$ 條專供調度卡車行駛的道路。每條道路連接兩個不同的站點，且卡車只能在這些道路上行駛。每條道路的長度皆為 1。這個站點網路是連通的，也就是說卡車可以在任何一對站點之間往返。每一晚，APIOBike 都必須派出調度卡車，以確保每個站點 i 最終都有恰好 $B[i]$ 輛單車。卡車可以從任意一個站點出發，也可以在任意一個站點結束行程。

在調度開始時，卡車是空的。每當卡車停在某個站點時，司機可以從該站點裝載任意數量的單車到卡車上，或從卡車上卸下任意數量的單車到站點。卡車和所有站點都可以容納無限多的單車，但任何地點的單車數量在過程中都不能變成負數。公司希望求出完成調度所需的最小行駛距離，以及對應的一種調度策略。

實作細節

你需要實作以下程序：

```
std::pair, std::vector>
find_rebalancing_strategy(int N,
                           std::vector A,
                           std::vector B,
                           std::vector U,
                           std::vector V)
```

- A ：長度為 N 的陣列，其中 $A[i]$ 表示每晚站點 i 上的單車數量。
- B ：長度為 N 的陣列，其中 $B[i]$ 表示每早目標是讓站點 i 擁有的單車數量。

- U, V : 長度為 $N - 1$ 的陣列，描述道路網路。對於每個 $0 \leq i < N - 1$ ，第 i 條道路連接站點 $U[i]$ 與 $V[i]$ 。
- 對於每個測試案例，這個程序 **最多會被呼叫 150 000 次**。

程序必須回傳兩個長度同為 $k + 1$ 的陣列 (X, Y) ，用以描述一個調度策略：

- X : 依時間順序列出卡車造訪的站點編號。
- Y : 在每個造訪站點的「淨交付單車數」。對於每個 $0 \leq j \leq k$:
 - 若 $Y[j] \geq 0$ ，則司機在站點 $X[j]$ 從卡車上卸下 $Y[j]$ 輛單車，該站點的單車數量增加 $Y[j]$ 。
 - 若 $Y[j] < 0$ ，則司機在站點 $X[j]$ 從該站點裝載 $-Y[j]$ 輛單車到卡車上，該站點的單車數量減少 $-Y[j]$ 。

一組合法的調度策略 (X, Y) ，其行駛距離為 k ，必須滿足以下條件：

- 對於每個 $0 \leq j \leq k$ ，都有 $0 \leq X[j] < N$ 。
- 對於每個 $0 \leq j < k$ ，站點 $X[j]$ 與 $X[j + 1]$ 之間必須有一條直接道路相連。
- 對於每個 $0 \leq j \leq k$ ，都必須滿足 $\sum_{t=0}^j Y[t] \leq 0$ （卡車上的單車數量永遠不為負）。
- 對於每個站點 $0 \leq i < N$ 和每個步驟 $0 \leq j \leq k$ ，令 $S_{i,j}$ 為所有滿足 $0 \leq t \leq j$ 且 $X[t] = i$ 的步驟之 $Y[t]$ 之總和，則必須滿足：
 - 任意時刻站點 i 的單車數量不會低於 0: $A[i] + S_{i,j} \geq 0$ 。
 - 在策略結束後，站點 i 的單車數量必須恰好等於 $B[i]$: $A[i] + S_{i,k} = B[i]$ 。

限制條件

- $2 \leq N \leq 300\,000$
- 在每個測試案例中，所有對 `find_rebalancing_strategy` 的呼叫裡， N 的總和不超過 300 000。
- 對於每個 $0 \leq i < N$ ，有 $0 \leq U[i], V[i] < N$ 且 $U[i] \neq V[i]$ 。
- 對於每個 $0 \leq i < N$ ， $0 \leq A[i], B[i] \leq 10^9$ 。
- 任意兩個站點之間皆可互相到達。
- $\sum_{i=0}^{N-1} A[i] = \sum_{i=0}^{N-1} B[i]$ 。
- 至少存在一個 $0 \leq i < N$ 使得 $A[i] \neq B[i]$ 。

子任務與配分

令 T 為呼叫 `find_rebalancing_strategy` 的次數， $\sum N$ 為所有呼叫中 N 的總和。

子任務	分數	額外限制條件
1	4	所有站點與道路滿足性質 P（見下）。 且恰好存在一個 $0 \leq i < N$ 使得 $A[i] > 0$ 。
2	11	$T \leq 10$ 、 $N \leq 7$ 。所有站點與道路滿足性質 P。
3	16	所有站點與道路滿足性質 P。
4	9	恰好存在一個 $0 \leq i < N$ 使得 $A[i] > 0$ 。
5	24	$\sum N \leq 500$ 。
6	15	$\sum N \leq 5000$ 。
7	21	無其他額外限制。

性質 P：對於每個 $0 \leq i < N - 1$ ，有 $U[i] = i$ 且 $V[i] = i + 1$ 。
對於每個 $0 \leq i < N$ ，都有 $A[i] \neq B[i]$ 。

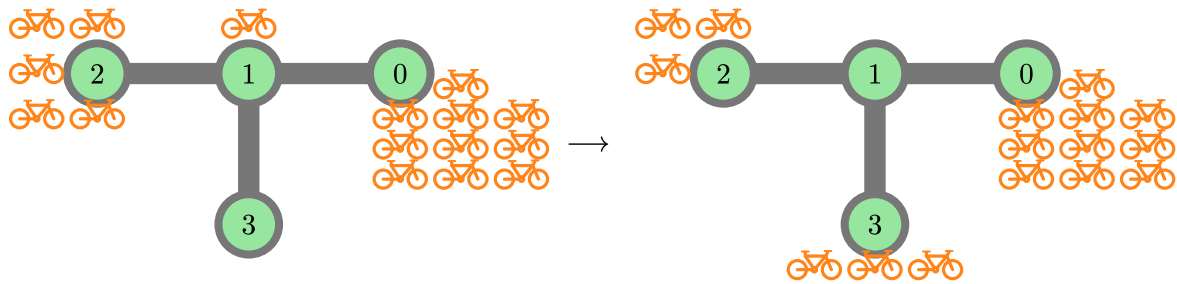
若回傳的陣列 X 與 Y 的長度**剛好**為 $k^* + 1$ ，其中 k^* 為理論上的最小行駛距離，但 (X, Y) 並不是一個合法策略，你將獲得該測試點分數的 50%。

範例

範例一

考慮以下呼叫：

```
find_rebalancing_strategy(4,
                        {10, 1, 5, 0},
                        {10, 0, 3, 3},
                        {0, 1, 1},
                        {1, 2, 3})
```



APIO 市有 $N = 4$ 個站點。

起初，各站點的單車數為 $A = [10, 1, 5, 0]$ ，目標是讓各站點變為 $B = [10, 0, 3, 3]$ 。

假設調度卡車從站點 2 出發，並依序執行下列步驟：

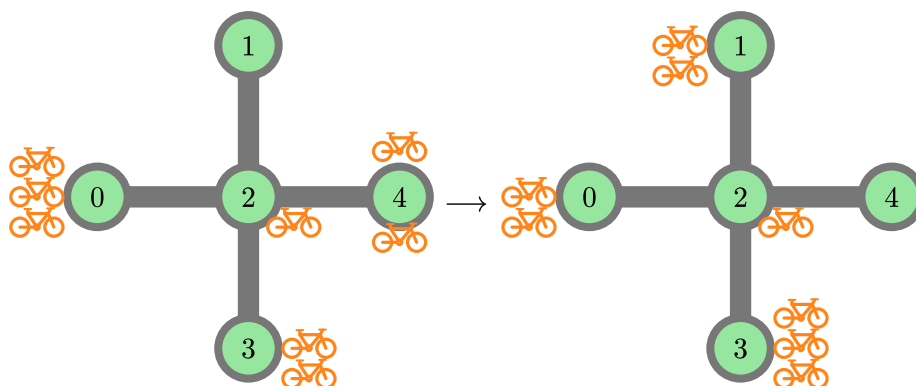
- 在站點 2 裝載 2 輛單車。此時站點 2 有 3 輛單車，卡車上有 2 輛單車。
- 行駛到站點 1，裝載 1 輛單車。此時站點 1 有 0 輛單車，卡車上有 3 輛單車。
- 行駛到站點 3，卸下 3 輛單車。此時站點 3 有 3 輛單車，卡車上有 0 輛單車。

總行駛距離為 2。這個策略對應的返回陣列為 $X = [2, 1, 3]$ 、 $Y = [-2, -1, 3]$ 。可以證明此策略達到最小行駛距離，因此程序應回傳 $([2, 1, 3], [-2, -1, 3])$ 。

範例二

考慮以下呼叫：

```
find_rebalancing_strategy(5,  
                           {3, 0, 1, 2, 2},  
                           {2, 2, 1, 3, 0},  
                           {2, 2, 2, 2},  
                           {0, 4, 3, 1})
```



APIO 市有 $N = 5$ 個站點。起初，各站點的單車數為 $A = [3, 0, 1, 2, 2]$ ，目標是 $B = [2, 2, 1, 3, 0]$ 。

假設調度卡車從站點 0 出發，並依序執行下列步驟：

- 在站點 0 裝載 1 輛單車。
- 行駛到站點 2，裝載 1 輛單車。
- 行駛到站點 1，卸下 2 輛單車。
- 行駛到站點 2。
- 行駛到站點 4，裝載 2 輛單車。
- 行駛到站點 2，卸下 1 輛單車。
- 行駛到站點 3，卸下 1 輛單車。

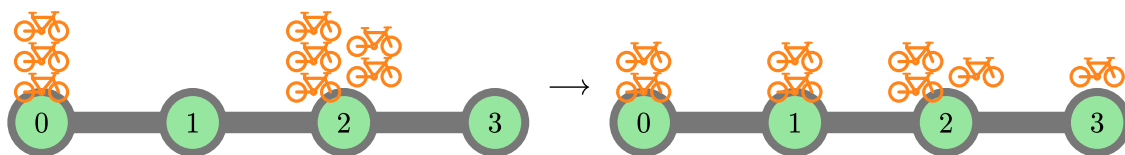
總行駛距離為 6。對應的返回陣列為 $X = [0, 2, 1, 2, 4, 2, 3]$ 、 $Y = [-1, -1, 2, 0, -2, 1, 1]$ 。可以證明此策略達到最小距離，因此程序應回傳 $([0, 2, 1, 2, 4, 2, 3], [-1, -1, 2, 0, -2, 1, 1])$ 。

其他同樣距離為 6 的合法策略，例如 $X = [0, 2, 3, 2, 4, 2, 1]$ 、 $Y = [-1, 0, 1, 0, -2, 0, 2]$ ，也都視為正確解。若回傳長度為 $7 = 6 + 1$ 但不是合法策略的陣列，例如 $X = Y = [0, 0, 0, 0, 0, 0, 0]$ ，則可得到該測試點分數的 50%。

範例三

考慮以下呼叫：

```
find_rebalancing_strategy(4,  
                           {3, 0, 5, 0},  
                           {2, 2, 3, 1},  
                           {0, 1, 2},  
                           {1, 2, 3})
```



一個最佳解為 $X = [2, 1, 0, 1, 2, 3]$ 、 $Y = [-1, 1, -1, 1, -1, 1]$ 。程序應回傳 $([2, 1, 0, 1, 2, 3], [-1, 1, -1, 1, -1, 1])$ 。其他合法策略，例如 $X = [2, 1, 0, 1, 2, 3]$ 、 $Y = [-2, 2, -1, 0, 0, 1]$ ，也同樣視為正確。

此範例符合子任務 2 與子任務 3 的限制。

範例評測器

輸入的第一行是一個整數 T ，表示情境數量。

接下來依序給出 T 個情境，每個情境的格式如下：

輸入格式：

```
N
A[0] A[1] ... A[N-1]
B[0] B[1] ... B[N-1]
U[0] V[0]
U[1] V[1]
...
U[N-2] V[N-2]
```

輸出格式：

```
k
X[0] X[1] ... X[k]
Y[0] Y[1] ... Y[k]
```